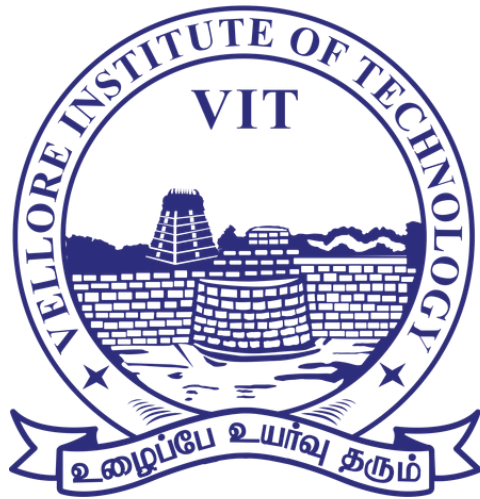




VIT[®]

BHOPAL

Adaptive Study Planner Agent

Project Report

Name: Varun Kumar

Reg No: 25MIM10071

Course: Fundamentals in AI and ML

Github Link: <https://github.com/var-0905/adaptive-study-planner>

1. Introduction

This report documents the Adaptive Study Planner Agent, a command line application that models a student's topic level mastery and produces a daily study plan using search over candidate study actions. The project was built for the Fundamentals of Artificial Intelligence and Machine Learning course and is meant to demonstrate, in a working system rather than slides, how an intelligent agent can combine a probabilistic model, a supervised learning component and an informed search algorithm to make a practical recommendation.

The system does not use any external AI service. All reasoning, from the Bayesian mastery update to the plan search, runs locally using standard Python and scikit-learn.

2. Problem Statement

Students preparing for several subjects at once usually decide what to study by feel rather than by any structured estimate of what they actually know. This means weak topics that need the most attention often get the least, especially once an exam date is close and time pressure sets in. The project addresses this by keeping an explicit, updatable estimate of mastery per topic, and using that estimate, along with difficulty, importance, exam urgency and time since last review, to decide what to study each day.

3. Objectives

- Represent a student's subjects and topics with the attributes needed for planning (mastery, difficulty, importance, exam date, history).
- Maintain an explainable Bayesian estimate of mastery, with an explicit uncertainty value, that updates from real assessment results.
- Use a small local classifier to predict how likely a study session on a topic is to succeed, and report genuine validation metrics for that classifier.
- Generate a study plan with an actual informed search algorithm driven by a transparent utility function, not a hand written priority list.
- Let the student record new results and see the plan and the underlying mastery values change.
- Expose why the plan looks the way it does, including which topics were considered and rejected.

4. Functional Requirements

4.1 Course and student management

Create a student profile with a name and daily available study minutes. Add subjects with an optional exam date. Add topics under a subject with a difficulty and importance rating.

4.2 Assessment and learning model

Record an assessment result (quiz, practice or exam) with a score between 0 and 1 for a specific topic. Update the Bayesian mastery estimate from that result. Keep a history of past results per topic. Use a trained classifier to predict the probability of a successful session on that topic.

4.3 Adaptive planning agent

Generate a ranked, time bounded study plan for the current day. Replan after new assessment results and show what changed compared to the previous plan. Respect the student's daily time budget and avoid scheduling the same topic too many times in one plan.

4.4 Analytics and explanation

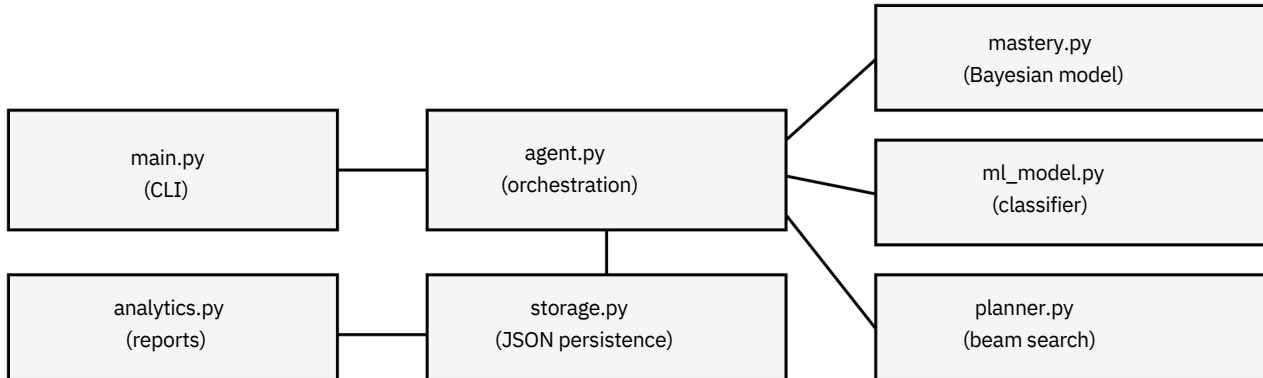
Show per topic progress (mastery, uncertainty, attempts, last studied date). List weak topics below a mastery threshold. List upcoming exams sorted by urgency. Show the mastery trend since the first recorded result for each topic. Explain the utility factors behind every chosen and top rejected topic in the latest plan.

5. Non Functional Requirements

Aspect	How it is addressed
Performance	Beam search keeps a small frontier for fast plan generation.
Usability	Menu-driven CLI with input validation and re-prompts.
Reliability	State is saved to JSON after each action.
Maintainability Error	Code is divided into small, focused modules.
Handling Resource	Invalid inputs are validated and handled safely.
Efficiency	Lightweight ML model runs locally without a GPU.

6. System Architecture

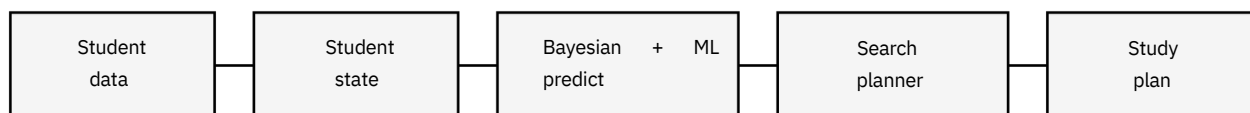
The application follows a simple layered structure. The CLI layer (main.py) handles input/output only. It calls into an agent layer that owns the planning workflow, which in turn uses three focused modules for the Bayesian mastery model, the ML predictor and the search based planner. Persistence and analytics sit alongside as independent modules.



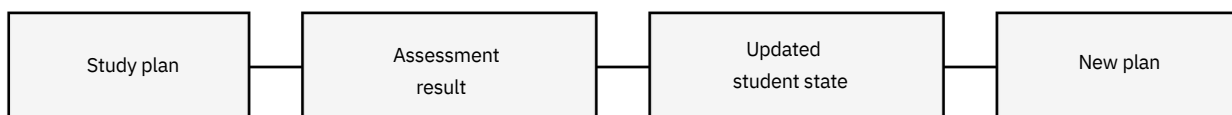
Data flows in one direction for reads: the CLI asks the agent for a plan, the agent asks the planner, the planner asks the ML predictor for a success probability per topic and reads mastery directly off the topic object. Writes (recording an assessment) go agent to mastery module, and the updated state is saved by storage.py after each CLI action.

7. Workflow / Process Diagram

The overall pipeline matches the one specified for the project: student data becomes a student state, which feeds the Bayesian mastery model and the ML predictor, which feed the search based planner, which produces a study plan. Recording an assessment result updates the state and triggers a replan.

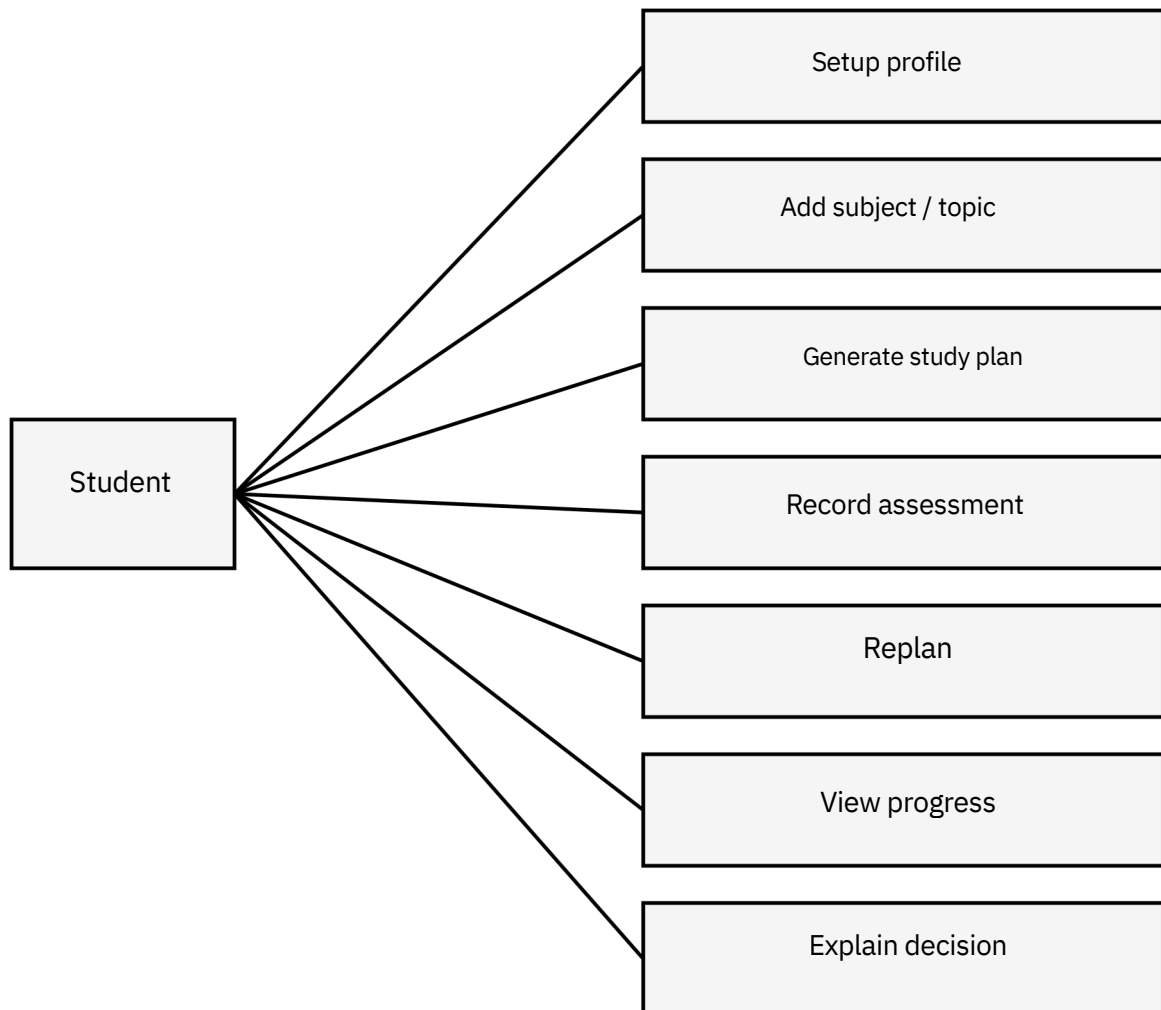


A second loop closes the cycle: Study plan leads to an Assessment result, which updates the Student state, which triggers the Bayesian and ML stage again and a new plan.



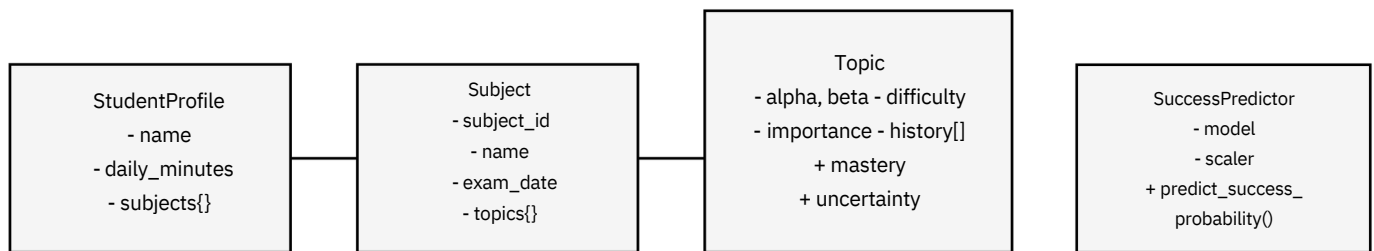
8. Use Case Diagram

The only actor is the student using the CLI. The use cases correspond directly to the menu options.



9. Class / Component Diagram

Simplified to the fields and relationships that matter for planning.



- BeamSearchPlanner and StudyPlannerAgent are not dataclasses; they hold behaviour.
- BeamSearchPlanner takes a SuccessPredictor and a StudentProfile and returns a list of Action objects.
- StudyPlannerAgent wraps BeamSearchPlanner, the mastery update functions, and exposes the operations the CLI calls directly.

10. Storage / Data Design

Persistence is plain JSON, chosen over SQLite because the data is a small nested tree (student -> subjects -> topics -> history) that maps directly onto JSON without needing relational joins, and because it keeps the project dependency free for storage.

data/profile.json	the student's live profile, saved after every menu action
data/demo_data.json	bundled read-only demo dataset (2 subjects, 6 topics, history)
data/training_data.csv	synthetic ML training data, generated on first run if absent

```
profile.json shape:
{
  "name": str,
  "daily_minutes": int,
  "subjects": {
    "<subject_id>": {
      "subject_id": str, "name": str, "exam_date": "YYYY-MM-DD" | null,
      "topics": {
        "<topic_id>": {
          "topic_id": str, "name": str, "subject_id": str,
          "difficulty": float, "importance": float,
          "alpha": float, "beta": float, "attempts": int,
          "last_studied": "YYYY-MM-DD" | null,
          "history": [ {"date": str, "score": float, "kind": str}, ... ]
        }
      }
    }
  }
}
```

11. Design Decisions and Rationale

- Beta-Bayesian mastery instead of a plain running average: it gives an uncertainty value for free, so the planner and the analytics view can tell the difference between 'confidently at 60 percent' and 'one lucky guess at 60 percent'.
- Logistic regression instead of a more complex model: the feature set is small and the relationship between features and success is expected to be roughly monotonic (more mastery, more recent success, less difficulty should raise the probability), so a linear-in-log-odds model is both sufficient and easy to explain through its coefficients.
- Beam search instead of a full exhaustive search or a greedy pick: a greedy pick can get stuck always picking the single highest utility topic every slot, since the repetition penalty is what prevents that, but a greedy method reacts to the penalty one step late. A full exhaustive search over action sequences grows quickly with the number of topics and slots. Beam search keeps a small set of the most promising partial plans at each step, which is a good middle ground for a CLI tool that has to feel instant.
- JSON over SQLite: the data is a small nested tree per single user, so a relational database adds complexity (schema, migrations, a query layer) without a real benefit at this scale.
- Synthetic training data instead of scraping or fabricating real results: the project explicitly should not invent real world claims, so the dataset is generated from a simple, documented formula with a fixed seed and is labelled as synthetic everywhere it is shown to the user.

12. AI/ML Methodology

This section covers the two learning components in the system and how the planner turns their output into a decision, following the intelligent agent framing used in the course: the agent perceives student state, and chooses actions to maximize a performance measure (importance weighted average mastery, shown under View progress) within the environment defined by exam dates and a daily time budget.

12.1 Bayesian mastery estimation

Each topic starts at $\alpha = 2$, $\beta = 2$, giving a neutral prior mastery of 0.5 with moderate uncertainty. When an assessment is recorded with score s (0 to 1):

```
if s >= 0.6 (pass threshold):  
    alpha += 0.5 + s  
else:  
    beta += 0.5 + (1 - s)
```

A strong pass (say 0.95) adds more to alpha than a narrow pass (0.62), and a strong fail adds more to beta than a narrow fail. Mastery is read as $\alpha / (\alpha + \beta)$; uncertainty is the standard deviation of the $\text{Beta}(\alpha, \beta)$ distribution, $\sqrt{\alpha\beta / ((\alpha+\beta)^2 * (\alpha+\beta+1))}$. As attempts accumulate, $\alpha + \beta$ grows, and the standard deviation shrinks, which is exactly the 'more evidence, less uncertainty' behaviour Bayesian updating is meant to give. This is verified directly in `test_mastery.py` (`test_uncertainty_decreases_with_more_attempts`).

A lightweight forgetting mechanism (`apply_forgetting`) pulls alpha and beta slightly back toward 1 the longer a topic has gone unstudied, on an exponential half life curve, so mastery estimates for topics the student has not touched in weeks drift back toward uncertainty rather than staying frozen at their last known value.

12.2 ML model and model selection rationale

The success predictor is a scikit-learn LogisticRegression trained on features standardized with StandardScaler. The seven input features are: current mastery, recent average score (last three results), latest score, topic difficulty, days since last study, attempts, and exam urgency (derived from days left before the subject's exam date). The target label is a synthetic binary 'success' outcome.

Logistic regression was chosen over alternatives such as random forests or gradient boosting because the project asks for an explainable model, and because the training set is small (a few hundred synthetic rows). A linear model's coefficients can be read directly as the direction and rough size of each feature's effect on the log odds of success, which is printed to the user through the 'Show ML model info' menu option. A more flexible model would risk overfitting a dataset this size and would not add any explanatory value the user could act on.

Dataset description: the synthetic dataset is generated by `src/ml_model.py` (`generate_synthetic_dataset`) with a fixed random seed. It draws feature values from simple distributions (mastery and exam urgency uniform on 0 to 1, days since study and attempts from bounded integer ranges) and derives a success probability from a weighted combination of the features passed through a logistic function, then samples the binary label from that probability. Sample size is 400 rows. This is a synthetic dataset built to give the classifier a plausible, documented relationship to learn; it is not drawn from or claimed to represent any real student population.

13.3 Model validation and actual results

The dataset is split 75/25 into train and test sets with stratification on the label, using scikit-learn's `train_test_split` with a fixed random state. The metrics below are the genuine output of running the project's test dataset generation and evaluation, not invented numbers.

Metric	Value
Train size	300
Test size	100
Accuracy	0.75
Precision	0.778
Recall	0.817
F1 score	0.797

Learned feature weights (on standardized features, so directly comparable in magnitude):

```
mastery: 0.685      recent_avg_score: 1.05
difficulty: -0.123   days_since_study: -0.81
exam_urgency: -0.193
```

Recent average score and current mastery have the largest positive weight, which matches the intuitive expectation that a student who has been doing well recently and already knows the topic reasonably well is more likely to have a successful next session. Days since last study has a negative weight, consistent with rustier topics being harder to succeed at immediately. The negative weight on exam_urgency reflects a pattern in the synthetic generation formula and should be read as a property of the training data, not a real claim that urgency itself makes a session harder; this is flagged here as a limitation of training on a synthetic formula rather than real behavioural data.

Bias and variance considerations: with only seven features and a linear model, the predictor sits on the higher-bias, lower-variance side, which is a reasonable choice for a small dataset since a higher variance model (for example an unpruned decision tree) would likely fit noise in only 300 training rows. The roughly 75 percent test accuracy is treated in the application as a moderately confident signal to blend into the planner's utility function, not as a precise probability.

13.4 Planning / search algorithm

The planner is a beam search over sequences of 20 minute study actions. The state is the remaining time budget, how many times each topic has already been used in this plan, and the actions chosen so far. At each step, every candidate (subject, topic) pair still under its repeat limit is scored with the utility function below, a new action is created, and the resulting state is added to the pool of candidates for the next step. The pool is sorted by total accumulated utility and trimmed to the beam width (default 4) before the next step. This bounds the search to beam_width x candidates work per step instead of growing combinatorially with the number of slots.

```
utility = 0.30 * mastery_gap
          + 0.20 * predicted_success    (from the ML model)
          + 0.20 * exam_urgency
          + 0.15 * importance
          + 0.15 * revision_need
          - 0.18 * repetition_count
```

The search stops once the remaining time in every surviving partial plan is below one slot, or after a maximum number of actions. The highest total utility plan in the final pool is returned along with a trace of the best utility seen at each step, which is shown to the user as the 'search trace' line.

14. Implementation Details

The project is implemented in plain Python 3 with scikit-learn and numpy as the only third party dependencies. Data classes (models.py) represent StudentProfile, Subject and Topic, each with a to_dict/from_dict pair for JSON serialization. The mastery module works directly on Topic objects and has no knowledge of the CLI. The ML predictor and the planner similarly take plain data (dicts or Topic/Subject objects) and have no direct dependency on the storage or CLI layers, which is what made the automated tests possible without touching the filesystem beyond a disposable CSV.

main.py contains only input handling, validation loops (read_float, read_int, read_date), and calls into the agent. All business logic - Bayesian updates, ML prediction, search - lives in src/.

15. Results and Screenshots

```
adaptive-study-planner on [~] main [X] via v3.14.7 (.venv)
$ python main.py
Loading Adaptive Study Planner...
ML model trained on synthetic data. Test accuracy: 0.75, f1: 0.797
```

```
Adaptive Study Planner
1. Setup profile
2. Add subject
3. Add topic
4. Generate study plan
5. Record assessment
6. Replan
7. View progress
8. Explain latest decision
9. Load demo dataset
10. Show ML model info
0. Exit
```

```
Choose an option: 9
Demo dataset loaded: 2 subjects, 6 topics, with existing history.
```

```
Adaptive Study Planner
1. Setup profile
2. Add subject
3. Add topic
4. Generate study plan
5. Record assessment
6. Replan
7. View progress
8. Explain latest decision
9. Load demo dataset
10. Show ML model info
0. Exit
```

```
Choose an option: 4
```

```
Generated study plan (80 minutes, 4 sessions):
```

1. [STUDY] Bayesian Reasoning (Computer Science) - 20 min | utility 0.645
2. [REVISE] Search Algorithms (Computer Science) - 20 min | utility 0.641
3. [STUDY] Calculus - Limits and Derivatives (Mathematics) - 20 min | utility 0.633
4. [PRACTICE] Supervised Learning Basics (Computer Science) - 20 min | utility 0.603

```
Search trace: explored 4 beam steps, final best utility 2.322
```

```
Adaptive Study Planner
1. Setup profile
2. Add subject
3. Add topic
4. Generate study plan
5. Record assessment
6. Replan
7. View progress
8. Explain latest decision
9. Load demo dataset
10. Show ML model info
0. Exit
```

```
Choose an option: 8
```

```
Why these topics were chosen:
```

```
Bayesian Reasoning (Computer Science) → study | utility 0.644
  mastery=0.34 gap=0.66 predicted_success=0.18 urgency=0.78 (exam in 10d) importance=0.8 revision_need=0.9 repetition_penalty=0.0
Calculus - Limits and Derivatives (Mathematics) → study | utility 0.633
  mastery=0.38 gap=0.62 predicted_success=0.53 urgency=0.62 (exam in 17d) importance=0.9 revision_need=0.53 repetition_penalty=0.0
Search Algorithms (Computer Science) → study | utility 0.605
  mastery=0.5 gap=0.5 predicted_success=0.86 urgency=0.78 (exam in 10d) importance=0.85 revision_need=0.0 repetition_penalty=0.0
Supervised Learning Basics (Computer Science) → practice | utility 0.603
  mastery=0.62 gap=0.38 predicted_success=0.86 urgency=0.78 (exam in 10d) importance=0.75 revision_need=0.33 repetition_penalty=0.0
```

```
Top topics considered but not selected:
```

```
Probability (Mathematics) | utility 0.519 | reasons {'mastery': 0.66, 'mastery_gap': 0.34, 'predicted_success': 0.93, 'exam_urgency': 0.62, 'days_to_exam': 17, 'importance': 0.7, 'revision_need': 0.0, 'days_since_study': 0, 'repetition_penalty': 0.0, 'utility': 0.519}
Matrices and Determinants (Mathematics) | utility 0.501 | reasons {'mastery': 0.5, 'mastery_gap': 0.5, 'predicted_success': 0.01, 'exam_urgency': 0.62, 'days_to_exam': 17, 'importance': 0.5, 'revision_need': 1.0, 'days_since_study': 999, 'repetition_penalty': 0.0, 'utility': 0.501}
```

```

adaptive-study-planner on / main [X1?] via v3.14.7 (.venv)
) python main.py
Loading Adaptive Study Planner...
ML model trained on synthetic data. Test accuracy: 0.75, f1: 0.797

Adaptive Study Planner
1. Setup profile
2. Add subject
3. Add topic
4. Generate study plan
5. Record assessment
6. Replan
7. View progress
8. Explain latest decision
9. Load demo dataset
10. Show ML model info
0. Exit

Choose an option: 5

1. Mathematics (3 topics)
2. Computer Science (3 topics)
Choose subject number: 1
1. Calculus - Limits and Derivatives (mastery 0.38)
2. Probability (mastery 0.75)
3. Matrices and Determinants (mastery 0.50)
Choose topic number: 2
Score achieved (0-1, e.g. 0.75 for 75%): 0.4
Assessment type (quiz/practice/exam) [practice]: exam

Updated mastery for 'Probability': mastery=0.657, uncertainty=0.151 (alpha=5.88, beta=3.08)

```

16. Testing Methodology

Tests are written with the standard library unittest module and organized by layer: test_mastery.py for the Bayesian update logic, test_planner.py for the utility function and the beam search, and test_agent.py for the orchestration layer and JSON persistence. The ML predictor is trained once per test class (setUpClass) against a disposable CSV path so tests do not depend on or modify the bundled dataset.

Coverage includes: correct update direction from pass/fail assessments, mastery bounds, uncertainty shrinking with more attempts, the forgetting mechanism pulling mastery back, exam urgency increasing as the exam date approaches, weak topics scoring a higher utility than strong ones, the repetition penalty actually reducing utility, the planner respecting the daily time budget, the planner handling an empty profile without crashing, replanning after new results, invalid subject/topic lookups raising a clear error, and a JSON save/load round trip.

Actual test run output (python -m unittest discover -s tests -v), 30 tests total.

17. Challenges

- Getting the beam search to behave correctly when a state has no remaining candidate actions (for example, a profile with only one topic that has already hit its repeat limit) required explicitly carrying that state forward instead of silently dropping it, which an early version of the planner did, leading to an empty search frontier and a crash. This was caught by the test suite (test_generate_plan_creates_actions) and fixed by tracking whether a state produced any expansion.
- Balancing the utility function weights so that no single factor (in particular exam urgency) dominates the others took a few iterations of running the demo dataset and checking that both the weakest topic

and the most urgent exam got reasonable priority.

- Deciding how much the forgetting mechanism should move mastery required care, since too aggressive a pull back makes long unstudied topics look artificially weak, and too weak a pull back makes the model ignore staleness entirely.

18. Learnings

- A small, explainable model (logistic regression) combined with a hand designed utility function can produce plans whose reasoning is easy to communicate to the end user, which matters more than squeezing out extra predictive accuracy for a tool like this.
- Keeping the ML and search modules free of any storage or CLI dependency made them directly testable and made it straightforward to reuse the same predictor instance across a whole CLI session instead of retraining it on every menu action.
- Bayesian updating gives a genuinely useful signal (uncertainty) that a plain score average would not, and it was cheap to implement once the Beta distribution mean/variance formulas were in place.

19. Future Enhancements

- Replace the flat forgetting decay with a proper spaced repetition schedule (for example SM-2 style).
- Expand the ML feature set and retrain once real (anonymized) usage data is available, instead of relying on the synthetic dataset.
- Show a multi day plan instead of a single day, with the search extended across days under a combined time and deadline budget.
- Add simple terminal based charts of mastery over time.
- Export the generated plan to a calendar (.ics) file.

20. References

- Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach - agents, informed search, and Bayesian reasoning concepts used as the basis for this project's design.
- scikit-learn documentation, LogisticRegression and model_selection modules (<https://scikit-learn.org/stable/>).
- Course material for Fundamentals of Artificial Intelligence and Machine Learning (VITyarthi).